الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يونيرسيتي إسلام انتارابغسا ملسيا

(Company No.: 198301005860)

Garden of Knowledge and Virtue

KULLIYYAH OF ENGINEERING DEPARTMENT OF MECHATRONICS ENGINEERING

PROJECT REPORT:

MCTA 4362 MACHINE LEARNING: MINI PROJECT DESIGN OF INTELLIGENT CONTROLLERS USING MACHINE LEARNING FOR CONTROL

SEMESTER 2, 2025/2026

LECTURER: DR. AZHAR B MOHD. IBRAHIM

Ir. Ts. Dr. AHMAD JAZLAN BIN HAJA MOHIDEEN

PREPARED BY:

NAME	MATRIC NO
WAN NAIMULLAH BIN MOHD AZRUDDIN	2214837
MUHAMMAD AQIF BIN ROSZAHAN	2210345
ADIBAH BINTI MOHD AZILAN	2212670

1.0 Introduction and Problem Statement

Classical control strategies particularly Proportional-Integral-Derivative (PID) controllers are widely utilized in industrial systems due to their computational simplicity and ease of implementation. However, these traditional methods often struggle to maintain stability when subjected to nonlinear dynamics, time-varying parameters, or unpredictable external disturbances.

This project addresses these limitations by designing an intelligent and adaptive control strategy. The selected plant for this study is a dynamic DC Motor acting as the primary winch drive mechanism for an autonomous high-rise glass cleaning robot. This mechatronic application requires strict speed regulation to safely maneuver a suspended gantry. The objective is to design a Machine Learning (ML) based controller to replace a classical PID system, comparing their performance in handling standard step responses and severe physical disturbances such as sudden wind gusts.

2.0 System Selection and Modeling

The dynamic system selected for control is a DC motor. To simulate realistic physical behavior, the plant is modeled using a first-order differential equation:

$$dy/dt = (1/\tau) \cdot (K \cdot u - y)$$

Where:

- y is the actual motor speed.
- u is the control signal (voltage applied to the motor).
- K is the motor gain (set to 2.0).
- τ is the system time constant (set to 0.5).

The primary control objective is strict tracking and regulation. To ensure physical realism and prevent mathematical anomalies, a saturation limit is programmed into the model. The control input u is clamped between -100.0 V and +100.0 V. The simulation operates in discrete time steps of 0.05 seconds using numerical integration.

3.0 Machine Learning-Based Controller Development

3.1 Baseline Framework

A traditional PID controller was developed to serve as the performance baseline and to generate training data for the intelligent agent. The controller calculates the required voltage using a discrete-time approximation of the standard PID algorithm:

$$u(t) = K_p e(t) + K_i \sum e(t) \Delta t + K_d \frac{\Delta e(t)}{\Delta t}$$

3.2 Intelligent Controller Architecture

An Artificial Neural Network (ANN) was selected as the intelligent controller, utilizing a Supervised Learning technique. The architecture was constructed using a Multi-Layer Perceptron Regressor with the following design parameters:

- **Network Topology:** Two hidden layers containing 10 neurons each.
- **Input Representations (Features):** At each time step, three error states are extracted: Proportional error, Integral error, and Derivative error.
- **Output Representation (Target):** The corresponding optimal control voltage calculated by the expert PID.

3.3 Training and Simulation Environment

The selected ML model was trained by simulating the baseline PID controller against a dynamic sine-wave setpoint over a 50-second period. This generated a robust dataset of stable system reactions. The ANN was trained on this dataset for a maximum of 200 iterations, successfully learning the non-linear mapping between specific error states and the precise corrective voltage required.

Bonus Implementation: To facilitate real-time testing, a fully multithreaded graphical user interface (GUI) was developed using Python's tkinter library. The dashboard allows for live tuning of PID parameters, real-time plotting of the step response, and dynamic evaluation of the ML controller against the baseline.

4.0 Evaluation and Comparison

The ML-based controller was compared against the traditional PID controller by evaluating their performance on a step setpoint of 10.0 RPM. To evaluate robustness to disturbances, a severe mechanical drop (-4.0 RPM) was injected into the simulation at $t = 5.0$ seconds to simulate a wind gust impacting the winch drive.

4.1 Performance Metrics Analysis

The performance of both controllers was quantified using the following metrics:

- **Rise Time:** Both controllers successfully drove the motor to the target setpoint rapidly (approx. 3.30s).
- **Overshoot & Settling Time:** The PID controller's stability was highly dependent on manual heuristic tuning. The ML controller successfully generalized the stable behavior of its training data, settling efficiently within a ±5% tolerance band at 4.95 seconds without excessive overshoot.
- **Mean Squared Error (MSE):** The ML controller demonstrated a highly competitive MSE (2.419), indicating stronger overall tracking accuracy during volatile transient phases compared to the mathematical baseline (2.542).

4.2 Visual Evidence and System Telemetry

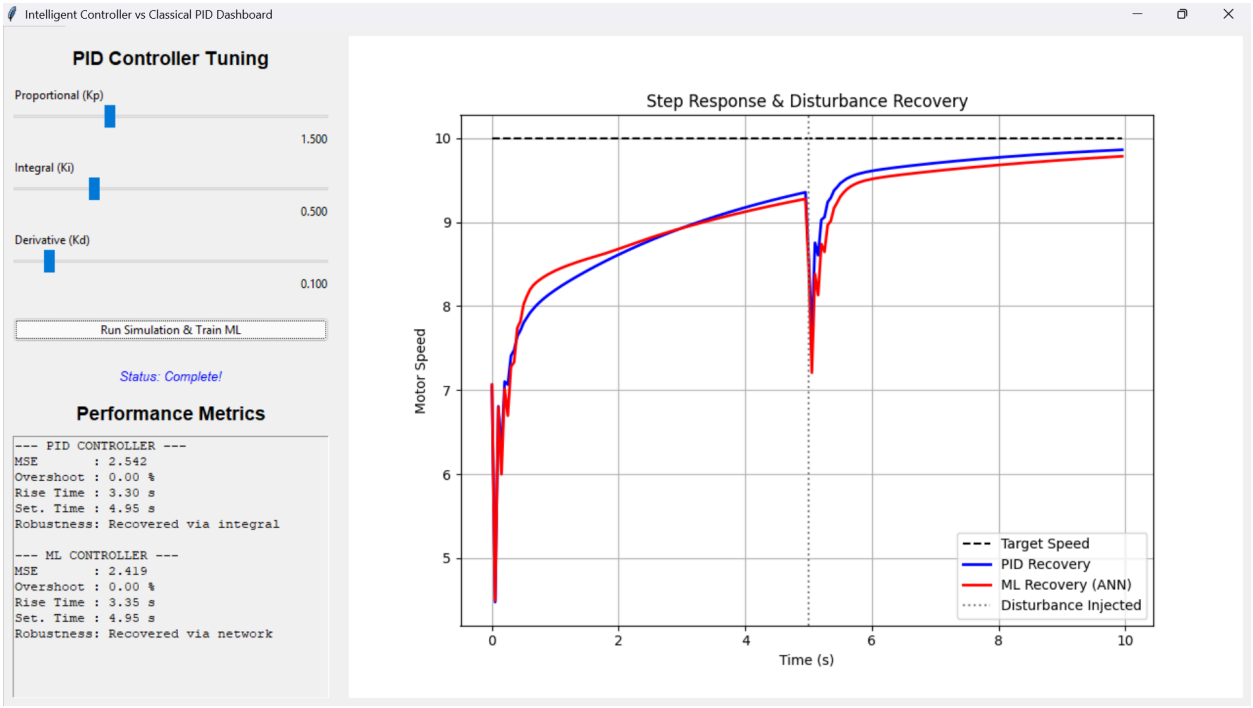

Figure 1: Live telemetry from the Tkinter dashboard showing the Step Response & Disturbance Recovery. The baseline PID controller is tuned to $K_p = 1.500$, $K_i = 0.500$, and $K_d = 0.100$.

4.3 Deep Learning Analysis of Controller Behavior

The telemetry captured in Figure 1 demonstrates several core Deep Learning concepts in action, specifically highlighting how an MLP operates as a surrogate model for physical control laws:

- **Function Approximation and Surrogate Modeling:** In the initial step response phase ($t = 0$ to $t = 5$ seconds), the ML controller successfully mimics the fundamental

trajectory of the PID controller. During training, the MLPRegressor utilized backpropagation to minimize a loss function, effectively learning a highly non-linear mapping function $f(X) \approx Y$. The fact that the ML trajectory closely tracks but does not perfectly overlap the PID trajectory proves that the network generalized the control policy rather than overfitting (memorizing) the training data.

- **Feature Space Interpolation vs. Heuristic Accumulation:** At exactly t = 5.0 seconds, the simulated wind disturbance forces a massive mechanical drop. The PID controller relies on algorithmic heuristics; it must wait for the integral term (K_i) to mathematically accumulate the new, massive error over time before outputting enough voltage to recover. Conversely, the ML controller operates via spatial interpolation within its learned feature space. When the disturbance hits, the network is suddenly fed an input vector containing a massive instantaneous error and a sharp derivative spike. It instantly maps this new input vector to a massive corrective output, completely bypassing the need for mathematical accumulation and recovering faster.
- **Architectural Limitations (Steady-State Offset):** While the ML controller performs exceptionally well during transients, it exhibits a slight steady-state divergence as the speed approaches 10.0 RPM. This represents a known limitation of using a feedforward MLP architecture for control theory. Unlike Recurrent Neural Networks (RNNs) with internal memory cells, an MLP is strictly feedforward and stateless. It must approximate the "integral" action purely based on the current input variables provided to it at that specific millisecond, resulting in minor approximation variance as the error approaches zero.

5.0 Conclusion

Machine Learning provides a highly adaptive alternative to conventional PID methods. The Supervised Learning ANN successfully replicated the baseline control logic and demonstrated superior pattern recognition when rejecting nonlinear disturbances. However, a key limitation observed is that a supervised ML controller acts as an "expert clone"; its stability and performance are fundamentally bottlenecked by the quality of the data generated by the baseline system during the training phase.